

词素 (Lexeme)	词法分析的基本单位,源程序中匹配某个模式的字符序列
词法单元 (Token)	词法分析器输出的抽象单元,由词素和附加信息组成
RE (正则表达式)	可以简洁、精确地描述词法单元的模式
DFA (确定有限自动机)	每个状态对每个输入符号只有唯一一个转换的自动机
NFA (非确定有限自动机)	一个状态对同一个输入符号可以有多个转换的自动机
ε -closure (ε 闭包)	从 NFA 某个状态集合出发,只通过 ε 转换能够到达的所有状态的集合
子集构造法	将 NFA 转换为 DFA 的算法, DFA 每个状态对应 NFA 一个状态集
Thompson 构造法	从正则表达式构造 NFA 的算法,按结构递归构造
左递归	文法中非终结符号 A 对某个串 α 存在推导 $A \Rightarrow A\alpha$
立即左递归	存在产生式 $A \rightarrow A\alpha$ 的左递归
自顶向下分析	从根结点开始向下到达叶子结点构造语法分析树的过程
递归下降分析	每个非终结符号对应一个过程的自顶向下分析方法
预测分析法	一种确定性的、无回溯的分析技术,通过查看下一个输入符号选择产生式
回溯	选择的产生式不匹配时退回到之前选择点尝试其他产生式
提取左公因子	消除文法中具有相同前缀的产生式的技术
FIRST(α)	可以从 α 推导得到的串的首符号的集合
FOLLOW(A)	可能在某些句型中紧跟在非终结符号 A 右边的终结符号的集合
LL(1) 文法	第一个 L 自左向右扫描,第二个 L 产生最左推导, 1 向前看一个符号
预测分析表	二维表 $M[A,a]$,告诉分析器非终结符 A、输入符号 a 时选择哪条产生式
文法	描述语言语法结构的形式规则系统
终结符号	文法中不能再被替换的基本符号
非终结符号	文法中可以被替换的符号
产生式	文法规则,形式为 $A \rightarrow \alpha$
推导	从开始符号出发反复应用产生式得到终结符号串的过程
最左推导	每一步都替换最左边的非终结符号的推导
句型	从开始符号推导得到的符号串(可包含非终结符号)

自底向上分析	从叶子结点开始向上构造语法分析树,最终到达根结点的语法分析过程
移入 - 归约	自底向上分析的核心操作,先将输入符号移入栈中,当栈顶形成句柄时进行归约
归约	将栈顶的句柄替换为对应的产生式左部非终结符号的操作
移进	将当前输入符号读取并压入分析栈的操作
最右推导	每一步都替换最右边的非终结符号的推导,自底向上分析对应最右推导的逆过程
句柄	句型的最左直接短语,是归约操作的核心对象,对应产生式的右部
移进 - 归约冲突	在 LR 分析中,同一状态下,既可以执行移进操作,又可以执行归约操作的冲突情况
归约 - 归约冲突	在 LR 分析中,同一状态下,存在多个可执行的归约操作的冲突情况
LR(k) 分析	一种自底向上的语法分析方法, L 表示自左向右扫描输入, R 表示构造最右推导的逆过程, k 表示向前看 k 个输入符号
LR(0) 项	带点的产生式,用于描述 LR 分析过程中已经识别的产生式部分,形式为 $A \rightarrow \alpha \cdot \beta$

规范 LR(0) 项集族	由LR(0)项通过闭包和GOTO操作构造得到的所有状态的集合，是LR(0)分析表的基础
增广文法	在原文法基础上新增一个开始符号S'和产生式S'→S，用于确保分析过程有唯一的接受状态
CLOSURE (项集闭包)	从给定的LR项集出发，通过添加所有可能的待约项得到的完整项集，是构造LR状态的核心操作
GOTO(I,X)	给定项集I和文法符号X，返回所有形如A→αXβ的项的闭包，用于描述状态之间的转换
内核项	项集中除了初始项S'→S之外，所有点不在产生式最左端的项，是项集的核心部分
非内核项	项集中点在产生式最左端的项（除初始项外），由内核项通过闭包操作生成
SLR (Simple LR)	简单LR分析方法，在LR(0)的基础上，通过FOLLOW集解决部分移进-归约冲突，降低了分析表的复杂度
SLR(1) 文法	可以通过SLR(1)分析方法进行无冲突分析的文法，要求对于同一状态下的归约项，其FOLLOW集与移进符号集不相交
LR(1) 文法	可以通过LR(1)分析方法进行无冲突分析的文法，每个LR(1)项包含向前看符号，能更精准地解决冲突，适用范围更广
LALR (Lookahead LR)	向前看LR分析方法，合并了LR(1)中同心的状态，在保持大部分LR(1)分析能力的同时，大幅降低了分析表的状态数
可归约流图	用于描述程序控制流的图结构，是代码优化和数据流分析的基础，由基本块和控制流边组成

3

语法制导定义 (SDD)	将语义规则附着到文法产生式上的形式系统, = 上下文无关文法 + 属性 + 语义规则
语法制导翻译 (SDT)	在产生式体中加入语义动作并在适当时执行
属性	与文法符号相关联的值,用来存储语义信息
属性文法	没有副作用的 SDD
综合属性	自下而上计算,从子结点传到父结点
继承属性	自上而下或兄弟结点之间传递
S 属性的 SDD	只包含综合属性的 SDD ,可按自底向上方式求值
L 属性的 SDD	每个属性要么是综合要么是继承,且计算满足特定约束
依赖图	描述分析树上各属性实例之间的信息流
注释语法分析树	包含各结点各属性值的语法分析树
语义规则	抽象层次上的定义,描述属性如何传播、计算和约束
语义动作	实现层次上的指令,嵌入产生式中的可执行代码
抽象语法树 (AST)	去除冗余语法细节,只保留核心结构的树
具象语法树 (CST)	按文法规则推导得到的完整树,精确反映文法层次
符号表	记录源程序中各种符号的数据结构,本质是名称 → 属性的映射表
作用域	标识符有效的程序区域
符号表栈	支持作用域嵌套的设计,进入新作用域新建,退出则弹出
类型检查	语义分析阶段检查类型是否匹配的过程
常量折叠	编译时计算常量表达式的值的优化技术

4

中间表示 (IR)	编译器在前端与后端之间使用的、与源语言和目标机器都相对独立的程序表示形式
高层 IR	接近源语言结构,便于高层语义相关优化
中层 IR	典型如三地址码,足够接近机器以便分析
低层 IR	显式寄存器 / 栈、分支、加载、存储,易映射到目标指令
三地址码 (TAC)	每条指令最多涉及三个 " 地址 " 的中间表示,格式: result = arg1 op arg2
四元式	三地址码实现方式,格式 (op, arg1, arg2, result)

三元式	三地址码实现方式,格式 (op, arg1, arg2) ,用位置引用结果
间接三元式	包含指向三元式的指针列表
静态单赋值形式 (SSA)	每个变量仅被赋值一次的中间表示,通过下标区分不同版本
Φ 函数	SSA 中在控制流合流点合并不同路径定值的伪指令
堆栈式 IR	操作数隐含在栈上的 IR (如 JVM 字节码)
布尔表达式	将布尔运算符作用在布尔变量或关系表达式上构成的表达式
关系表达式	形式为 $E_1 \text{ rel } E_2$ 的表达式
短路求值	布尔表达式求值时,一旦确定整个表达式的结果就停止后续子表达式的求值
回填 (Backpatching)	先生成带占位符的跳转指令,后续再填充实际标签的延迟绑定技术
truelist	布尔表达式取值 true 时执行的跳转指令位置列表
falselist	布尔表达式取值 false 时执行的跳转指令位置列表
nextlist	语句执行完毕后紧接着执行的跳转指令位置列表
穿越 (Fall-through)	不生成跳转指令,顺序进入下一条指令
基本块	中间代码的最大直线指令序列,控制流只能从第一个指令进入
流图 (控制流图)	结点是基本块,边指明执行顺序的图

5

静态区	大小在编译时已知的数据对象的存放区域
栈区	存放活动记录的数据结构,栈向较低地址方向增长
堆区	存放生命周期不确定的数据对象的区域
活动记录 (栈帧)	过程调用和返回由控制栈管理,每个活跃活动对应栈中一个活动记录
活动树	表示程序运行过程中过程调用序列的树
控制栈	管理过程调用和返回的栈
栈底指针 (BP)	指向栈帧底部的寄存器
栈顶指针 (SP)	指向栈顶的寄存器
返回地址	函数调用时保存的调用点之后的指令地址
存储管理器	分配 / 回收堆区空间的子系统
碎片	堆区逐渐被割裂成空闲存储块和已用存储块的交错
窗口 (Hole)	堆中的空闲存储块
Best-Fit	将请求分配在满足请求的最小窗口中的策略
First-Fit	将对象放置在第一个能容纳请求的窗口中的策略
垃圾	不能被引用(不可达)的数据
垃圾回收	自动回收不可达数据的机制
可达性	存储块可以被程序访问到的性质
根集	不需要指针解引用就可以直接访问的数据
增变者	用户程序,修改堆中的对象集合
引用计数	每个对象有引用计数字段,为 0 则回收的垃圾回收方法
标记 - 清扫式垃圾回收	从根集标记可达对象,遍历堆释放不可达对象的算法
标记并压缩垃圾回收	在标记清扫基础上对可达对象重定位以消除碎片

拷贝回收器	堆分为两个半空间,回收时可达对象拷贝到另一半空间
半空间	拷贝回收器中堆被分成的两份相邻空间(fromspace 和 tospace)
分代回收	将堆区切割成块并标号,较年轻的世代更频繁被回收
记忆集	分代回收中存储高号堆块对低号堆块引用的集合
Stop-the-world	GC 期间整个系统被挂起的现象
空闲空间列表	包含堆中所有未分配存储块的列表
边界标记	标识存储块空闲 / 已用状态和大小的标记

6

指令选择	把 IR 程序映射成为目标机上可运行代码序列的过程
指令代价	每个指令的固定代价,设定为 1 加上运算分量寻址模式的代价
寄存器分配	确定哪些变量值应该存放在寄存器中的过程
寄存器指派	确定哪个变量使用哪个具体寄存器的过程
加载指令 (LD)	把地址 addr 中的内容加载到寄存器
保存指令 (ST)	把寄存器中的内容保存到内存
MAXLIVE	每条指令处存活值的最大数量
溢出 (Spill)	寄存器不够用时将部分值保存到内存中的操作
后续使用信息	变量在后续指令中是否会被使用的信息
活跃变量	在程序执行完语句 i 后, x 中存放的值将被后面的语句使用
寄存器描述符	跟踪各个寄存器都存放了哪些变量的当前值
地址描述符	某个变量的当前值存放在哪个或哪些位置
getReg 函数	根据描述符和数据流信息为三地址指令选择最佳寄存器的函数
DAG (有向无环图)	基本块的图表示,每个变量有对应结点代表初始值
死代码	DAG 上没有附加活跃变量的根结点对应的代码
入口结点	流图中额外添加的结点,到第一条指令有一条边
出口结点	流图中额外添加的结点,从可能最后执行的基本块到此有边
前驱 / 后继	有向边的起点是终点的前驱,终点是起点的前驱
循环入口	循环中唯一的、前驱可以在循环外的结点

7

代码优化	消除不必要指令,把指令序列替换为更快的指令序列
数据流分析	获取数据沿着程序执行路径流动的信息的技术
数据流抽象	把程序状态集合总结为一些特性的方法
程序点	三地址语句之前或之后的位置
执行路径	从 p_1 到 p_2 的程序点序列
传递函数	基于语句语义的约束, $IN[s]=f_s(OUT[s])$ 或 $OUT[s]=f_s(IN[s])$
控制流方程	基于控制流的约束, $IN[s_{i+1}]=OUT[s_i]$
到达定值	如果存在从定值 d 后面到达点 p 的路径且 d 没被杀死,则 d 到达 p
杀死 (Kill)	路径上对 x 的其他定值杀死了之前的定值
生成 (Gen)	语句生成了对某个变量的定值

交汇运算符	到达定值中为 $\cup$, 可用表达式中为 $\cap$
活跃变量分析	分析变量在 p 上的值是否会在某条从 p 出发的路径中使用
use 集合	在基本块 B 中引用但引用前没有被定值的集合
def 集合	在基本块中定值但定值前没有被引用的变量集合
可用表达式	从入口到 p 的每条路径都对 $x+y$ 求值且之后没对 x 或 y 赋值
公共子表达式消除	表达式之前已计算过且运算分量值没变, 则重用之前结果
复制传播	在 x 的应用点 u 可以用 y 替代 x (存在 $x=y$ 且之后无重新定值)
死代码消除	消除不会影响程序结果的代码
支配结点 (必经节点)	每一条从入口到达 n 的路径都经过 d , 则 d 支配 n
后支配	$A \text{ post-dom } B$, 所有路径从 B 到 Exit 都经过 A
严格支配	$A \text{ strict-dom } B$, 但 $A \neq B$
直接支配	$A \text{ strict-dom } B$, 但不存在 C 使得 $A \text{ strict-dom } C$ 且 $C \text{ strict-dom } B$
支配结点树	表示支配关系的树, 根结点是入口结点
深度优先排序	对流图进行深度优先搜索时的节点排序
DFST (深度优先生成树)	对流图进行深度优先搜索得到的生成树
自然循环	有唯一入口结点支配循环中所有结点的最大强连通分量
循环头	自然循环的唯一入口结点
回边	边 $a \rightarrow b$, 头 b 支配了尾 a
强连通分量	任意两个节点都互相可达的最大子图
Pre-Header	循环头之前的结点, 用于插入代码
最内层循环	不包含其它循环的循环
最小不动点	迭代解法最终收敛到的解
Worklist	迭代算法中维护的工作列表
部分冗余消除	消除在部分路径上重复计算的表达式的优化技术